

Assignment - 09

Title : Heap Sort

Objective : To study some advanced data structure.

Problem Statement :

Implement Heap Sort to sort given set of values using max or min heap.

Outcome :- Solve problem using algorithm design techniques and data structure.

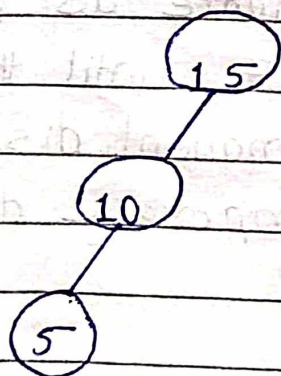
Theory :

Heap :- Heap is a Complete Binary tree or almost Complete Binary tree in which every parent node is either greater or lesser than its child node.

Heap can be min heap or max heap.

- Max heap :- A Max heap is a tree in which value of each node is greater than or equal to the value of its children nodes.

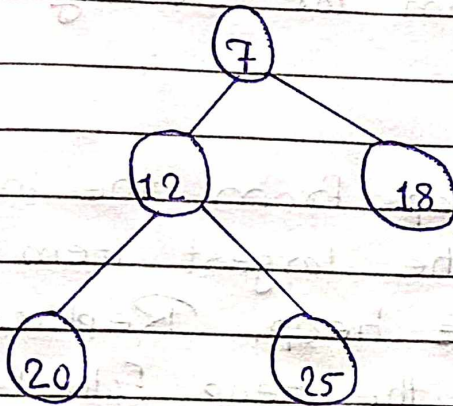
e.g :-



Note that every parent node is greater than its children.

- Min heap :- A Min heap is a tree in which value of each node is less than or equal to value of its children nodes.

e.g :-



Note that every parent node is less than its children.

Feature's of Heap Sort :-

1. The time Complexity of heap sort is $O(n \log n)$.
2. It does not require extra storage space while sorting the elements.
3. The Space Complexity of Heap sort is $O(1)$. As it does not require any extra storage to sort.

• Time Complexity of heap sort is $O(n \log n)$ in both worst and average case.

Heap can be implemented as array by recording its element in the top-down and left to right fashion.

e.g :-

For array
 $A[\text{Size}]$

→ Root node is stored at $A[0]$.

→ Left child is at $2i + 1$ position in array A .

→ Right child is at $2i + 2$ position in array A .

Algorithm :-

i). Heap Sort Algorithm For Sorting in Increasing order :-

- 1). Build a max heap from the input data.
- 2). At this point, the largest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of heap by 1. Finally heapify the root of the tree.
- 3). Repeat step 2 while the size of the heap is greater than 1.

ii). Heap sort Using min heap :-

1. Build a min heap from the input data.
2. At this point, the smallest item is stored at the root of the heap. Replace it with the last item of the heap followed by reducing the size of heap by 1. Finally heapify the root of the tree.
3. Repeat above steps while size of heap is greater than 1.

Test Case	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
1.	Check result when user select an option of heap sort using Max heap.	Heap sort the element in an array using Max heap	Sorting of element get done using Max heap	Element in an array get sorted using max heap	pass
2.	Check result when user select an option of Heap sort using Min heap	Heap sort the element in an array using min heap	Element should get Sorted using min heap	Element get's Sorted using min heap in an array	pass

```
#include <iostream>

// #define SIZE 100

using namespace std;

void min_heap(int arr[], int n, int i)
{
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;
    if (l < n && arr[l] > arr[largest])
        largest = l;
    if (r < n && arr[r] > arr[largest])
        largest = r;
    if (largest != i) {
        swap(arr[i], arr[largest]);
        min_heap(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        min_heap(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
```

```

        swap(arr[0], arr[i]);
        min_heap(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << "\n";
}

int main()
{
    int arr[6];

    cout<<"Enter numbers::"<<endl;
    for (int i = 0; i < 6; i++)
    {
        cin>>arr[i];
    }

    int n = sizeof(arr) / sizeof(arr[0]);

    heapSort(arr, n);

    cout << "Sorted array is \n";
    printArray(arr, n);
}

```